

NumPy

Complete Student Roadmap

All 16 topics · 80+ functions · ML/DS/AI ready

16 Total topics	80+ Functions & methods	10 Must-learn topics
---------------------------	-----------------------------------	--------------------------------

● Must learn (10 topics)

● Important (5 topics)

● Nice to know (1 topic)

Student Action Plan

Phase 1 — Week 1-2

Topics 1 to 5

Array creation, attributes, data types, indexing & slicing, math operations.

Phase 2 — Week 3-4

Topics 6 to 10

Aggregation & stats, shape manipulation, boolean indexing, broadcasting, sorting.

Phase 3 — Week 5-6

Topics 11 to 16

Linear algebra, memory, vectorization, ML utilities, set ops, saving & loading.

Golden rule: Don't just read — open a Jupyter notebook and run every single function yourself. Type it, break it, fix it. That's how it sticks.

Priority Overview

#	Topic	Priority
1	Array Creation Functions	Must learn
2	Array Attributes & Inspection	Must learn
3	Data Types & Type Casting	Must learn
4	Indexing & Slicing	Must learn
5	Array Operations & Math	Must learn
6	Aggregation & Statistics	Must learn
7	Shape Manipulation	Must learn
8	Boolean Indexing & Filtering	Must learn
9	Broadcasting	Must learn
10	Sorting & Searching	Must learn
11	Linear Algebra (np.linalg)	Important
12	Copying & Memory	Important
13	Vectorization & Performance	Important
14	ML-Specific Utility Functions	Must learn
15	Set Operations	Nice to know
16	Saving & Loading Data	Important

All 16 Topics in Detail

1 Array Creation Functions

Must learn

BASIC

`np.array()`

`np.zeros()`

`np.ones()`

`np.full()`

`np.eye()`

`np.empty()`

RANGES

`np.arange()`

`np.linspace()`

`np.logspace()`

RANDOM

`np.random.rand()`

`np.random.randn()`

`np.random.randint()`

`np.random.seed()`

`np.random.choice()`

`np.random.shuffle()`

LIKE-ARRAY

`np.zeros_like()`

`np.ones_like()`

`np.full_like()`

Note: Foundation of everything — learn this first.

2 Array Attributes & Inspection

Must learn

SHAPE & SIZE

`.shape`

`.ndim`

`.size`

`.dtype`

`.itemsize`

`.nbytes`

INSPECT

`np.info()`

`type(arr)`

`print(arr)`

Note: Always check shape and dtype when debugging ML code.

3 Data Types & Type Casting

Must learn

COMMON DTYPES

`int8`

`int32`

`int64`

`float32`

`float64`

`bool`

`complex`

CASTING

`.astype()`

`np.float32()`

`np.int64()`

Note: float32 vs float64 matters a lot in deep learning memory usage.

4 Indexing & Slicing

Must learn

BASIC

`arr[0]`

`arr[-1]`

`arr[1:5]`

`arr[:,2]`

2D / 3D

`arr[0, 1]`

`arr[0:2, 1:3]`

`arr[:, 0]`

FANCY

`arr[[0,2,4]]`

`arr[np.array([1,3])]`

Note: Multi-dimensional slicing is used constantly in image/tensor data.

5 Array Operations & Math

Must learn

ELEMENT-WISE

`+`

`-`

`*`

`/`

`**`

`//`

`%`

MATH FUNCTIONS

`np.sqrt()`

`np.exp()`

`np.log()`

`np.log2()`

`np.log10()`

`np.abs()`

`np.ceil()`

`np.floor()`

`np.round()`

MATRIX

`np.dot()`

`@`

`np.matmul()`

`np.cross()`

`np.outer()`

Note: `np.dot()` and `@` are the core of all neural network forward passes.

6 Aggregation & Statistics

Must learn

BASIC STATS

`np.sum()`

`np.mean()`

`np.median()`

`np.min()`

`np.max()`

`np.std()`

`np.var()`

ADVANCED

`np.percentile()`

`np.quantile()`

`np.average()`

`np.corrcoef()`

`np.cov()`

CUMULATIVE

`np.cumsum()`

`np.cumprod()`

AXIS PARAM

`axis=0` (column-wise)

`axis=1` (row-wise)

`axis=-1` (last dim)

Note: The `axis=` parameter is tested in almost every ML interview.

7 Shape Manipulation

Must learn

RESHAPE

`reshape()`

`flatten()`

`ravel()`

`.T`

`transpose()`

EXPAND / SQUEEZE

`np.expand_dims()`

`np.squeeze()`

COMBINE

`np.concatenate()`

`np.stack()`

`np.hstack()`

`np.vstack()`

`np.dstack()`

SPLIT / REPEAT

`np.split()`

`np.hsplit()`

`np.vsplit()`

`np.tile()`

`np.repeat()`

Note: `reshape()` and `expand_dims()` used daily in ML for batch dimensions.

8 Boolean Indexing & Filtering

Must learn

FILTERING

`arr[arr > 5]`

`arr[arr == 0]`

`arr[(a>2) & (a<8)]`

WHERE / ANY / ALL

`np.where()`

`np.any()`

`np.all()`

NAN HANDLING

`np.isnan()`

`np.isinf()`

`np.isfinite()`

`np.nan_to_num()`

Note: `np.nan_to_num()` is critical for cleaning real-world ML datasets.

9 Broadcasting

Must learn

CONCEPT

shape alignment rules

rank stretching

automatic expansion

PRACTICAL USE

adding bias term

row normalization

scalar ops on arrays

Note: Understanding broadcasting = understanding how TensorFlow/PyTorch work internally.

10 Sorting & Searching

Must learn

SORT

`np.sort()`

`np.argsort()`

`np.lexsort()`

SEARCH

`np.argmin()`

`np.argmax()`

`np.searchsorted()`

`np.nonzero()`

UNIQUE

`np.unique()`

`np.unique(return_counts=True)`

Note: `argmax()` used in every classifier to get the predicted class label.

1
1

Linear Algebra (np.linalg)

Important

CORE

np.linalg.norm()

np.linalg.dot()

np.linalg.inv()

np.linalg.det()

DECOMPOSITION

np.linalg.eig()

np.linalg.svd()

np.linalg.qr()

SOLVE

np.linalg.solve()

np.linalg.lstsq()

np.linalg.matrix_rank()

Note: SVD and eig() needed to truly understand PCA.

1
2

Copying & Memory

Important

VIEW VS COPY

arr.view()

arr.copy()

arr.base

MEMORY

.flags

np.shares_memory()

np.may_share_memory(
)

Note: Forgetting .copy() causes silent bugs in ML preprocessing pipelines.

1
3

Vectorization & Performance

Important

VECTORIZE

np.vectorize()

np.frompyfunc()

np.apply_along_axis()

BEST PRACTICE

avoid Python loops

use broadcasting
instead

use ufuncs

Note: Vectorized code is 10-100x faster than Python for loops.

1
4

ML-Specific Utility Functions

Must learn

CLIPPING & SIGNS

np.clip()

np.sign()

np.abs()

ACTIVATION MATH

np.tanh()

np.sinh()

np.cosh()

DISTRIBUTION/FREQ

np.histogram()

np.bincount()

np.digitize()

Note: `np.clip()` used in gradient clipping; `np.sign()` in perceptrons.

1
5

Set Operations

Nice to know

SET OPS

np.union1d()

np.intersect1d()

np.setdiff1d()

np.isin()

Note: Useful for comparing label sets and finding common features.

1
6

Saving & Loading Data

Important

NUMPY FORMAT

np.save()

np.load()

np.savez()

np.savez_compressed()

TEXT / CSV

np.loadtxt()

np.savetxt()

np.genfromtxt()

Note: `.npy` is fastest format for saving preprocessed arrays between sessions.